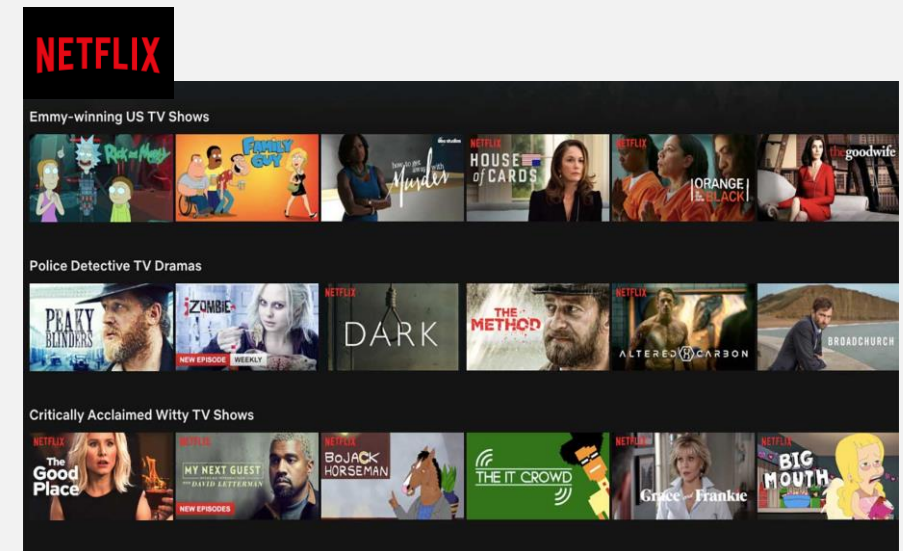
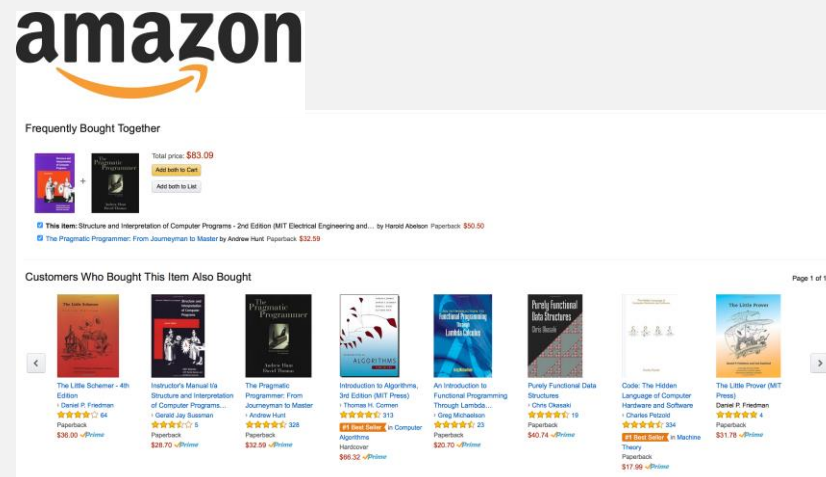


RECOMMENDER SYSTEMS

DEFINITION

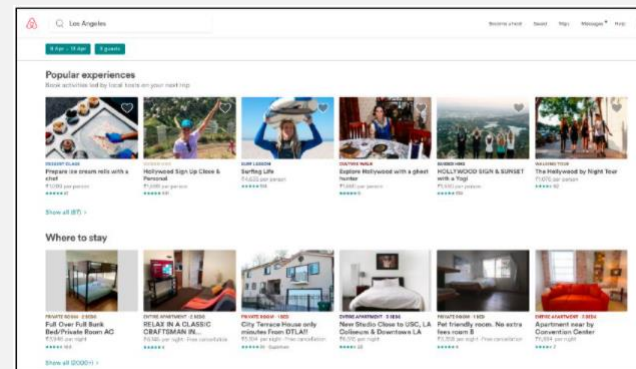
- A recommender system is a specific form of information filtering to present the information elements (movies, music, books, news) that are likely to interest the user.
- Recommender systems are pretty self-explanatory; as the name suggests, they are systems or techniques that recommend or suggest a particular product, service, or entity.



RECOMMENDATIONS APPROACHES



	i_1	i_2	i_3	i_4	i_5	i_6
j_1	4	?	3	?	5	?
j_2	?	2	?	?	4	1
j_3	?	?	1	?	2	5
j_4	?	?	3	?	?	1
j_5	1	4	?	?	2	5
j_6	5	?	2	1	?	4
j_7	?	2	3	?	4	5



RECOMMENDATIONS APPROACHES

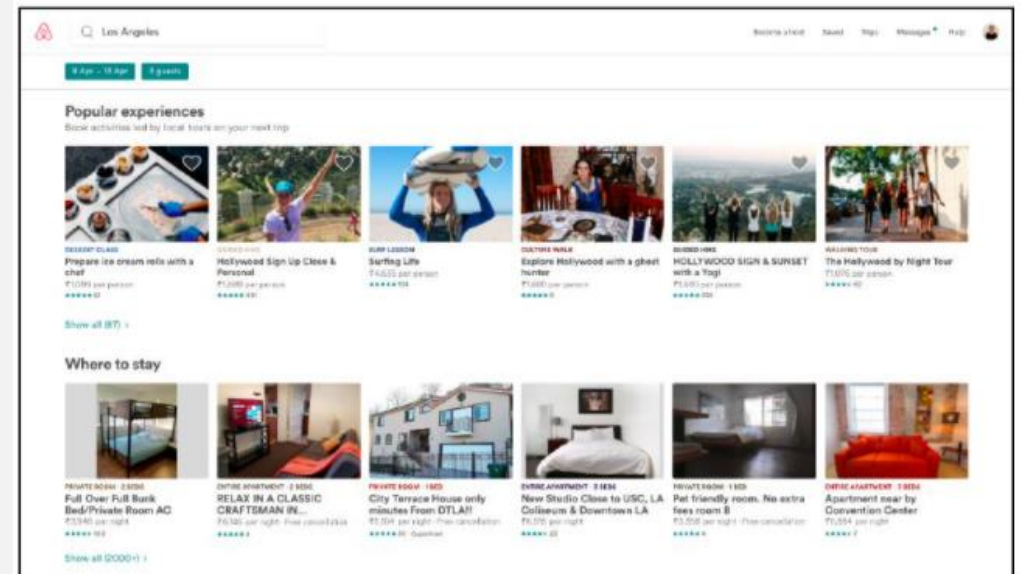
- **Prediction problem**

- In this version of the problem, we are given a matrix of m users and n items. Each row of the matrix represents a user and each column represents an item. The value of the cell in the i^{th} row and the j^{th} column denotes the rating given by user i to item j . This value is usually denoted as r_{ij} .
- **Objective** : Predict missing values using all the information it has at its disposal (the ratings recorded, data on movies, data on users, and so on).

	i_1	i_2	i_3	i_4	i_5	i_6
u_1	4	?	3	?	5	?
u_2	?	2	?	?	4	1
u_3	?	?	1	?	2	5
u_4	?	?	3	?	?	1
u_5	1	4	?	?	2	5
u_6	5	?	2	1	?	4
u_7	?	2	3	?	4	5

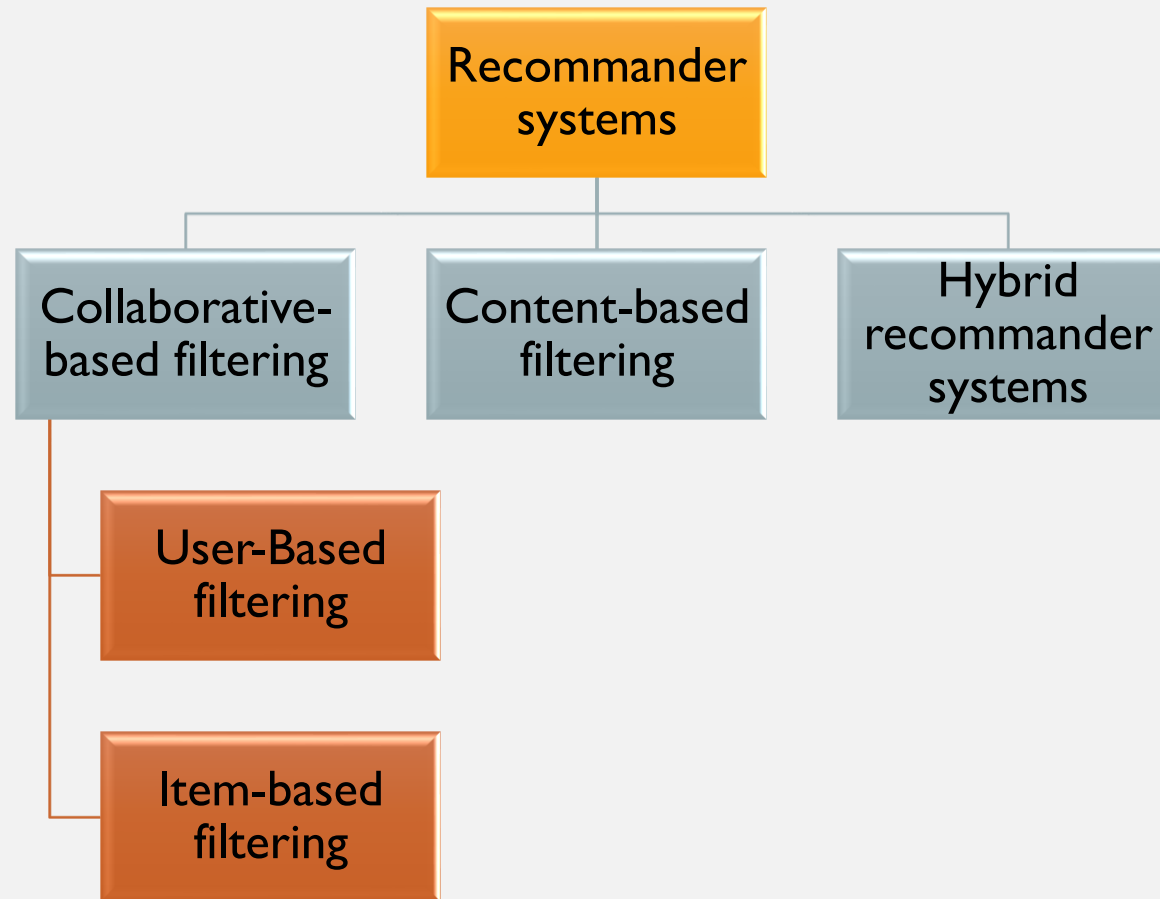
RECOMMENDATIONS APPROACHES

- **Ranking problem**
- We are given a matrix of n items
- **Objective** : Tries to discern the top k items to recommend to a particular user, utilizing all of the information at its disposal.



RANKING PROBLEM

TYPES



COLLABORATIVE-BASED FILTERING

ALGORITHMS: COLLABORATIVE-BASED MODEL

Collaborative Filtering:

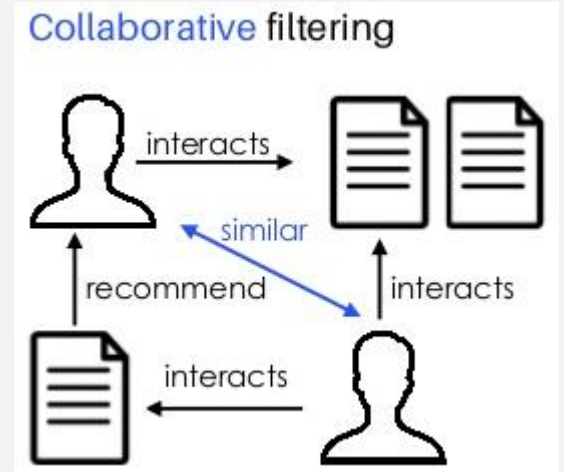
- Looks for patterns in the user activity to produce user specific recommendations;
- Depends only on usage data (e.g. rating, purchases, downloads, user preferences, ...).

Example:

- User ratings on books they have read indicating how much they liked them.

Key idea:

- The rating of a user for a new item is likely to be similar to that of another user, if both users have rated other items in similar way.
==> No dependence on having any additional information about the **items** (e.g. description, metadata, etc) or the **users** (e.g. interests, demographic data, etc)



ALGORITHMS: COLLABORATIVE-BASED MODEL

What is a similarity metric ?

- In statistics and related fields, a **similarity measure** or **similarity function** is a real-valued function that quantifies the similarity between two objects. (Wikipedia)

Variants ?

- Euclidian distance
- Pearson correlation
- Cosine similarity
- Jaccard similarity
- ...

ALGORITHMS: COLLABORATIVE-BASED MODEL

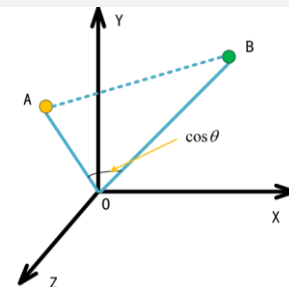
What is a cosine similarity ?

- Cosine similarity is a commonly used similarity measure for real-valued vectors, used in (among other fields) information retrieval to score the similarity of documents in the vector space model. (Wikipedia)

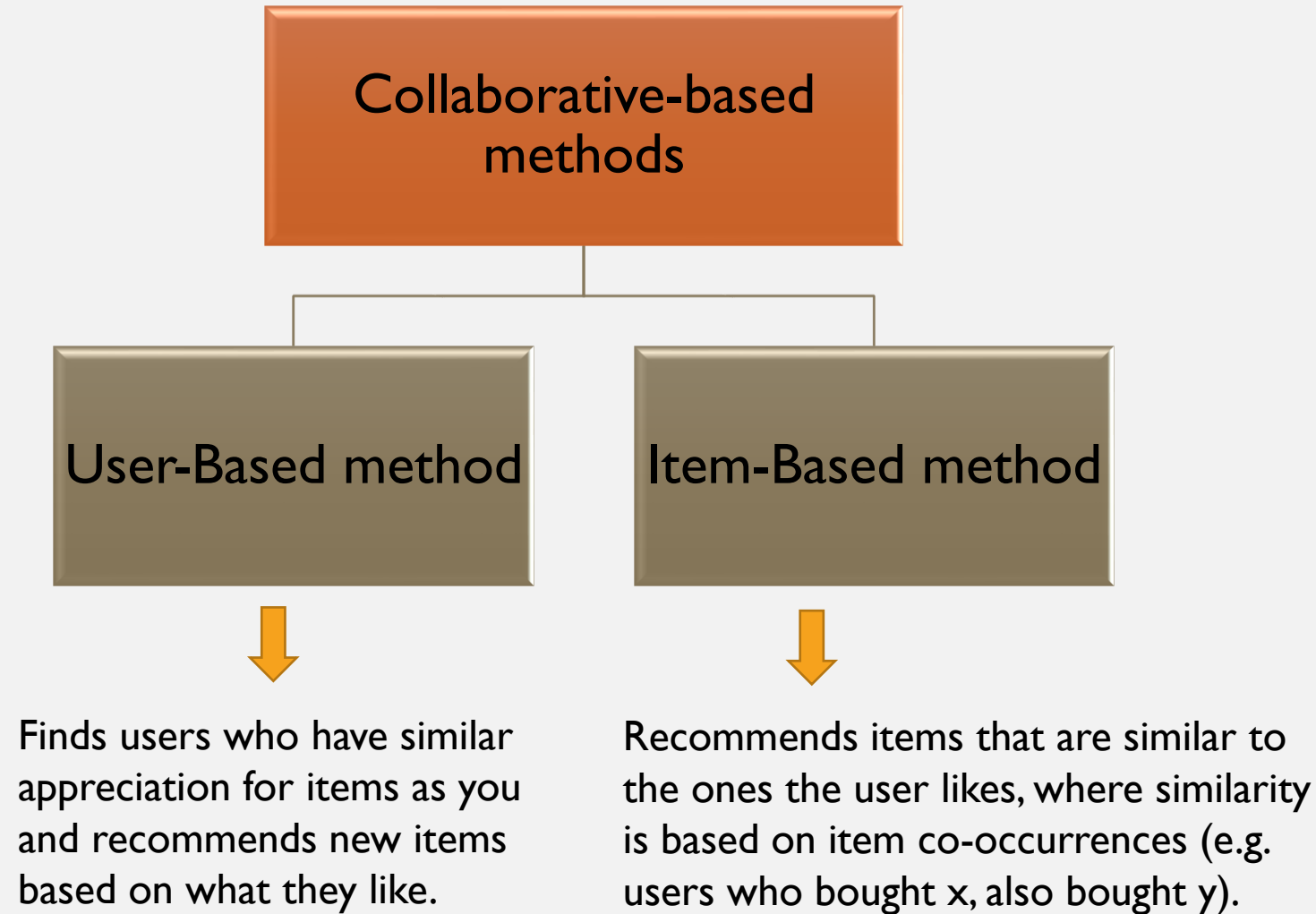
How to calculate a cosine similarity ?

Given two vectors $A = [a_1, a_2, \dots, a_n]$ and $B = [b_1, b_2, \dots, b_n]$ with n attributes each, the cosine similarity, $\cos(\theta)$ is represented as

$$\cos(\theta) = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \times \sqrt{\sum_{i=1}^n b_i^2}}$$



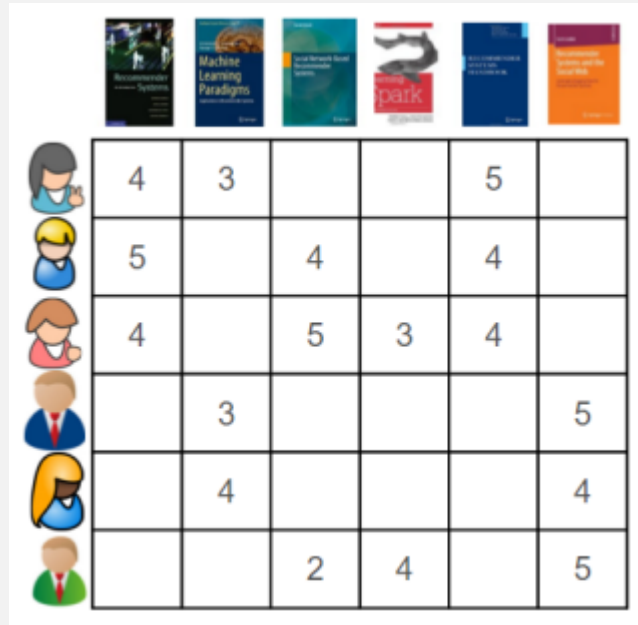
ALGORITHMS: COLLABORATIVE-BASED MODEL



ALGORITHMS: COLLABORATIVE-BASED MODEL

➡ User-based method

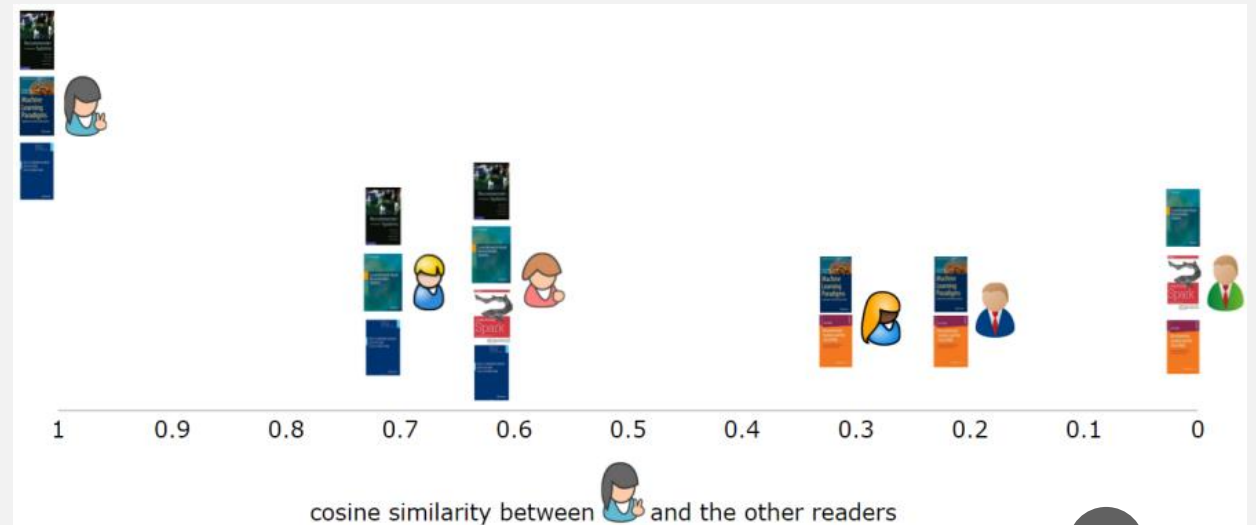
User-based method: Example



	Recommendation Systems	Machine Learning Paradigms	Data Mining Introductory Texts	The Park	Introduction to Data Mining	Introduction to Data Mining
User 1	4	3			5	
User 2	5		4		4	
User 3	4		5	3	4	
User 4		3				5
User 5		4				4
User 6			2	4		5

- Figure showing users preferences for books
- The more that they like the book, the higher a rating they give it, from a scale of one to five.

1. Calculate how similar users are to one another based on their preferences for the books.
2. Variety of similarity metrics ➡ **cosine similarity**.
3. Take the first user and compare it to the five other users.
4. The first user is **quite similar** to two users, as they share two books in common, **less similar** to two other users, who share just one book in common and **not at all similar** to the last user with whom they share no books in common.



ALGORITHMS: COLLABORATIVE-BASED MODEL

➔ User-based method

Back to the example ...

Using user-based collaborative filtering

Let's calculate how similar each user is to all users and represent them in a similarity matrix !

According to the previous matrix:

- $V_{user1} = (4, 3, 0, 0, 5, 0)$
- $V_{user2} = (5, 0, 4, 0, 4, 0)$
- $V_{user3} = (4, 0, 5, 3, 4, 0)$
- $V_{user4} = (0, 3, 0, 0, 0, 5)$
- $V_{user5} = (0, 4, 0, 0, 0, 4)$
- $V_{user6} = (0, 0, 2, 4, 0, 5)$

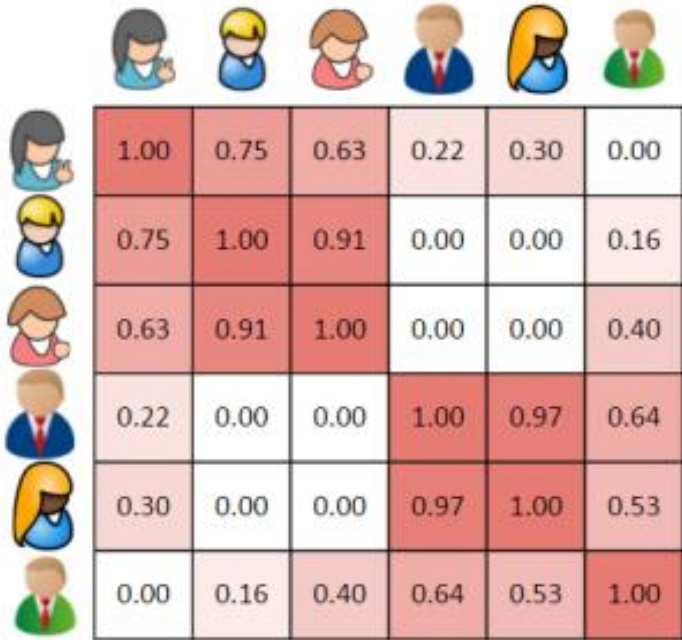
$$\text{Cos}(V_{user1}, V_{user2}) = V_{user1} \cdot V_{user2} / |V_{user1}| \cdot |V_{user2}|$$

$$V_{user1} \cdot V_{user2} = 20 + 20 = 40$$

$$|V_{user1}|^2 = 16 + 9 + 25 = 50$$

$$|V_{user2}|^2 = 25 + 16 + 16 = 57$$

➔ $\text{Cos}(V_{user1}, V_{user2}) = 0.75$



						
	1.00	0.75	0.63	0.22	0.30	0.00
	0.75	1.00	0.91	0.00	0.00	0.16
	0.63	0.91	1.00	0.00	0.00	0.40
	0.22	0.00	0.00	1.00	0.97	0.64
	0.30	0.00	0.00	0.97	1.00	0.53
	0.00	0.16	0.40	0.64	0.53	1.00



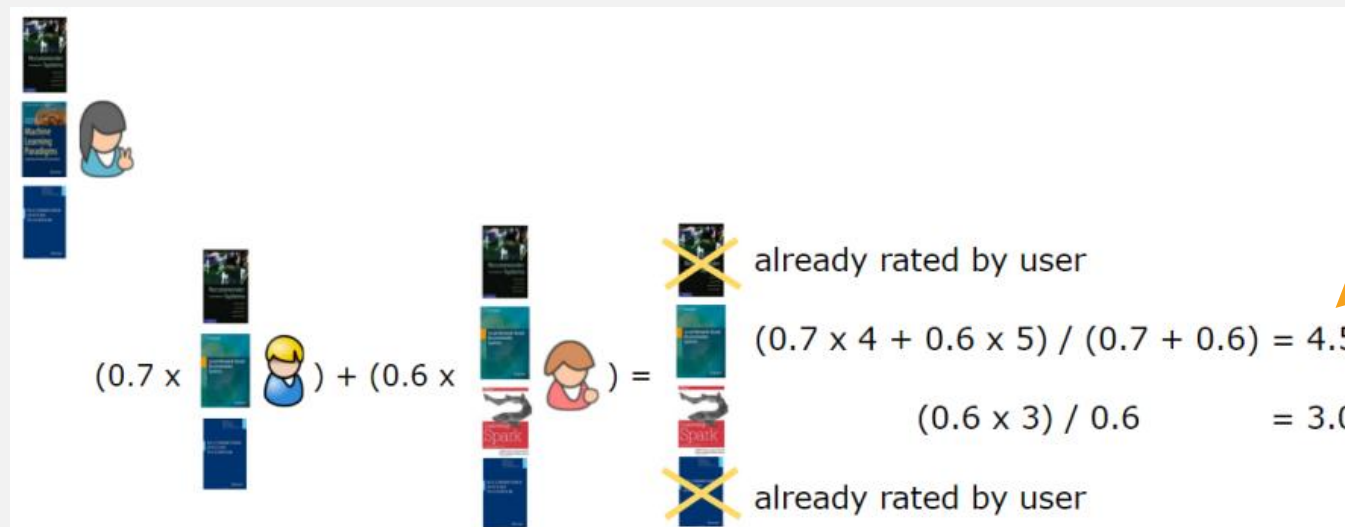
ALGORITHMS: COLLABORATIVE-BASED MODEL

→ User-based method

But , what about recommendations for users ?

1. Finding the top n users who are most similar to a given user i ;
2. Finding the users who are most similar to him;
3. Recommending the items that these similar users appreciate, weighting them by how similar the users are.
 1. Remove books that the user has already given preferences for;
 2. Weight the books that the most similar users are reading, and sum them together.

Example: $n = 2$



Recommended items

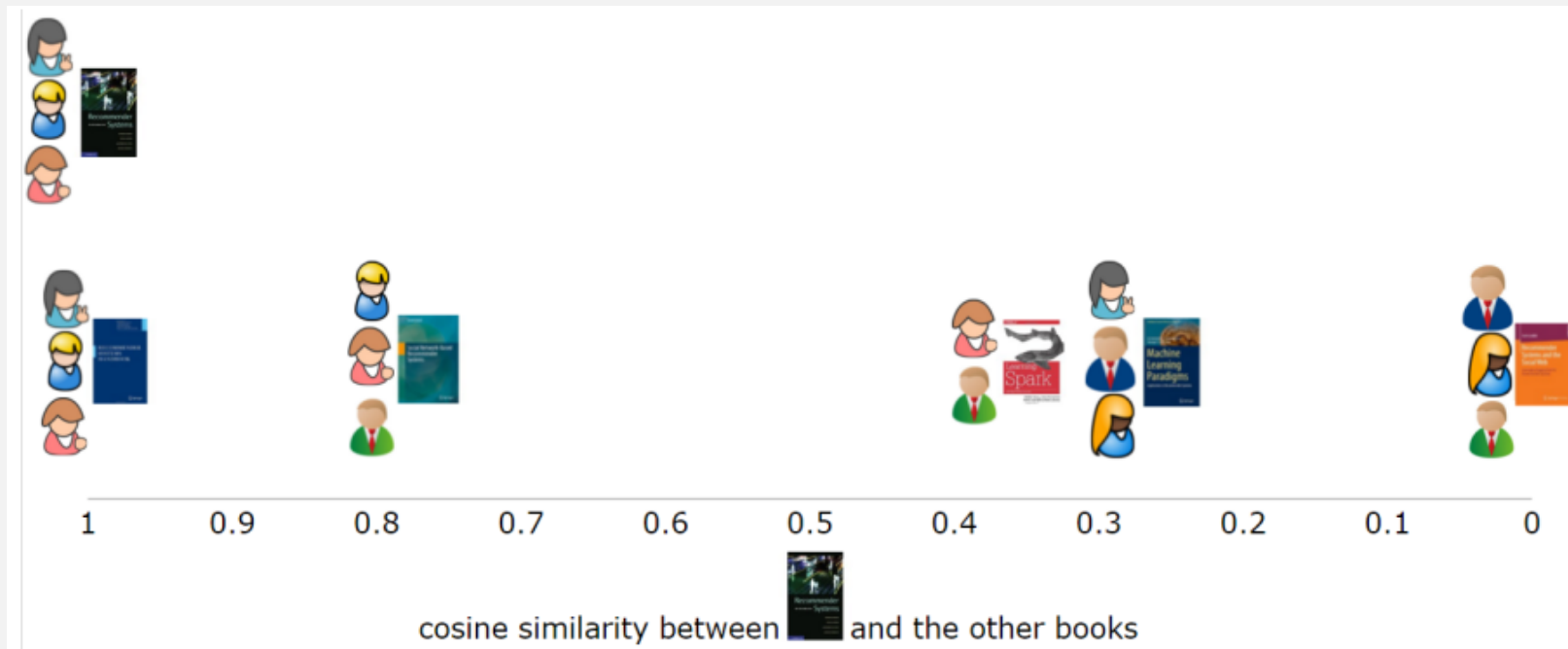


ALGORITHMS: COLLABORATIVE-BASED MODEL

→ Item-based method

Now, how it works with Item user-based collaborative filtering ?

1. Similar to user-based, the first thing is to calculate a similarity matrix but with respect to **items** rather than users.
2. Represent books as vectors (or arrays) of users who appreciate them;
3. Apply cosine similarity function.



ALGORITHMS: COLLABORATIVE-BASED MODEL

→ Item-based method

Now, how it works with Item user-based collaborative filtering ?

- $V_{\text{book1}} = (4, 5, 4, 0, 0, 0)$
- $V_{\text{book2}} = (3, 0, 0, 3, 4, 0)$
- $V_{\text{book3}} = (0, 4, 5, 0, 0, 2)$
- $V_{\text{book4}} = (0, 0, 3, 0, 0, 4)$
- $V_{\text{book5}} = (5, 4, 4, 0, 0, 0)$
- $V_{\text{book6}} = (0, 0, 0, 5, 4, 5)$

1.00	0.27	0.79	0.32	0.98	0.00
0.27	1.00	0.00	0.00	0.34	0.65
0.79	0.00	1.00	0.69	0.71	0.18
0.32	0.00	0.69	1.00	0.32	0.49
0.98	0.34	0.71	0.32	1.00	0.00
0.00	0.65	0.18	0.49	0.00	1.00

- Let consider the first user;
- Look for books that are not rated by him. In the example, concerned books are: 3rd, 4th and 6th.
- Find among them the top n books which are most similar to the rated ones.
 - The 3rd one (similar to the 1st and the 5th)
 - The 6th one (similar to the 2nd)
- Weight the selected most similar books and sum them together by applying the function:



set of the similar items

$$\sum_1$$

$Rate_{user} * similarity$

Recommended items

ALGORITHMS: COLLABORATIVE-BASED MODEL

User-based Vs Item-based collaborative filtering

- User-based and Item-based collaborative filtering sound very similar to one another, but they can generate different results.
- Both of them enjoy considerable popularity due to their simplicity and efficiency, and their ability to produce accurate and personalized recommendations.
- They have some scalability limitations as they require a similarity computation (between users or items) that grows with both the number of users and the number of items.



CONTENT FILTERING

ALGORITHMS: CONTENT-BASED MODEL

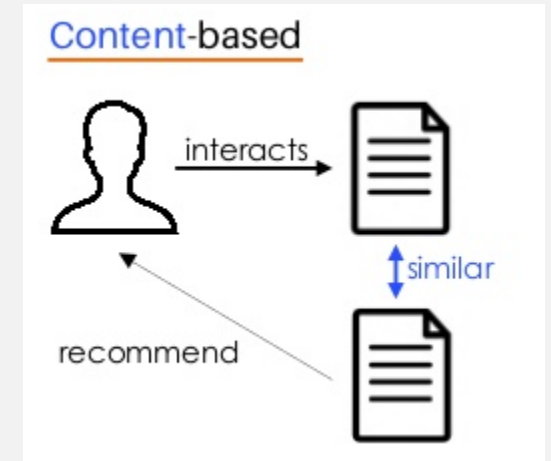
Content Filtering:

- Recommends items with similar content (metadata, description, topics) to the items the user has liked in the past;
- Depends only on the content/descriptions of the items and users (but not usage data).
- Content-based filtering recommends items that are similar to the ones the user liked in the past, by deriving the similarity between items based on their content (e.g. title, year, description) and not how people use them.
- Most content-based recommendation techniques were applied to recommending text documents (webpages, newgroup messages, ...etc.)

Example:

If a user likes

- “Lord of the Rings: The Fellowship of the Ring”
- “Lord of the Rings: Two Towers”
- → using the words in the title, the recommender may suggest “**Lords of the Rings: The Return of the King**”.

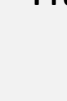


ALGORITHMS: CONTENT-BASED MODEL

Let's see its main steps on the same example:

						
	4	3			5	
	5		4		4	
	4		5	3	4	
		3				5
		4				4
			2	4		5

User preferences for books.

	Introduction to Recommender Systems
	Machine learning Paradigms
	Social Network-based Recommender Systems
	Learning Spark
	Recommender Systems Handbook
	Recommender Systems and the Social Web

Titles of the books users have rated.

ALGORITHMS: CONTENT-BASED MODEL

Let's see its main steps on the same example:

1. Remove the stop words (e.g. grammar words, very common words) from the content → It's common to do that !
2. Represent the books as a vector that indicates which words are present in the title: **vector-space representation**.

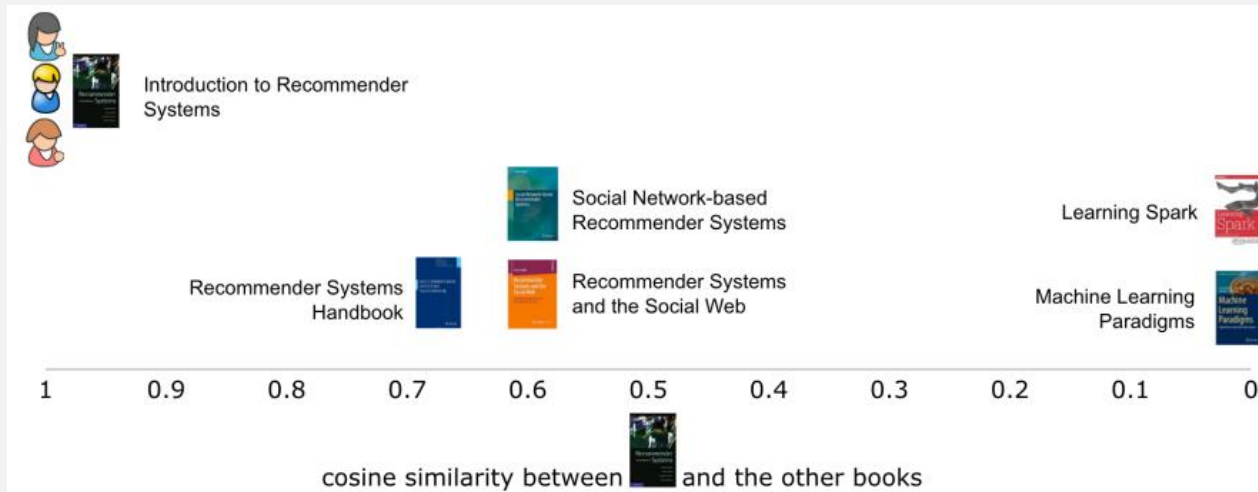


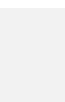
introduction	1					
recommender	1		1		1	1
systems	1		1		1	1
machine		1				
learning		1		1		
paradigms		1				
social			1			1
network-based			1			
spark				1		
handbook					1	
web						1

ALGORITHMS: CONTENT-BASED MODEL

Let's see its main steps on the same example:

3. Use cosine similarity as for item-based method. The difference is that here, the similarity is based on the content of the books.



						
	1.00	0.00	0.58	0.00	0.67	0.58
	0.00	1.00	0.00	0.41	0.00	0.00
	0.58	0.00	1.00	0.00	0.58	0.75
	0.00	0.41	0.00	1.00	0.00	0.00
	0.67	0.00	0.58	0.00	1.00	0.58
	0.58	0.00	0.75	0.00	0.58	1.00

4. Take the books that a user has previously rated, and recommend other books that are most similar to them.



ALGORITHMS: CONTENT-BASED MODEL

Let's see its main steps on the same example:

$(4 \times \text{book cover}) + (3 \times \text{book cover}) + (5 \times \text{book cover}) = \text{book cover} \text{ (already rated by user)} \quad (0.4 \times 3) / 0.4 = 3.0$

$\text{book cover} \text{ (already rated by user)} \quad (0.6 \times 4 + 0.6 \times 5) / (0.6 + 0.6) = 4.5$

ALGORITHMS: CONTENT-BASED MODEL

Limitations:

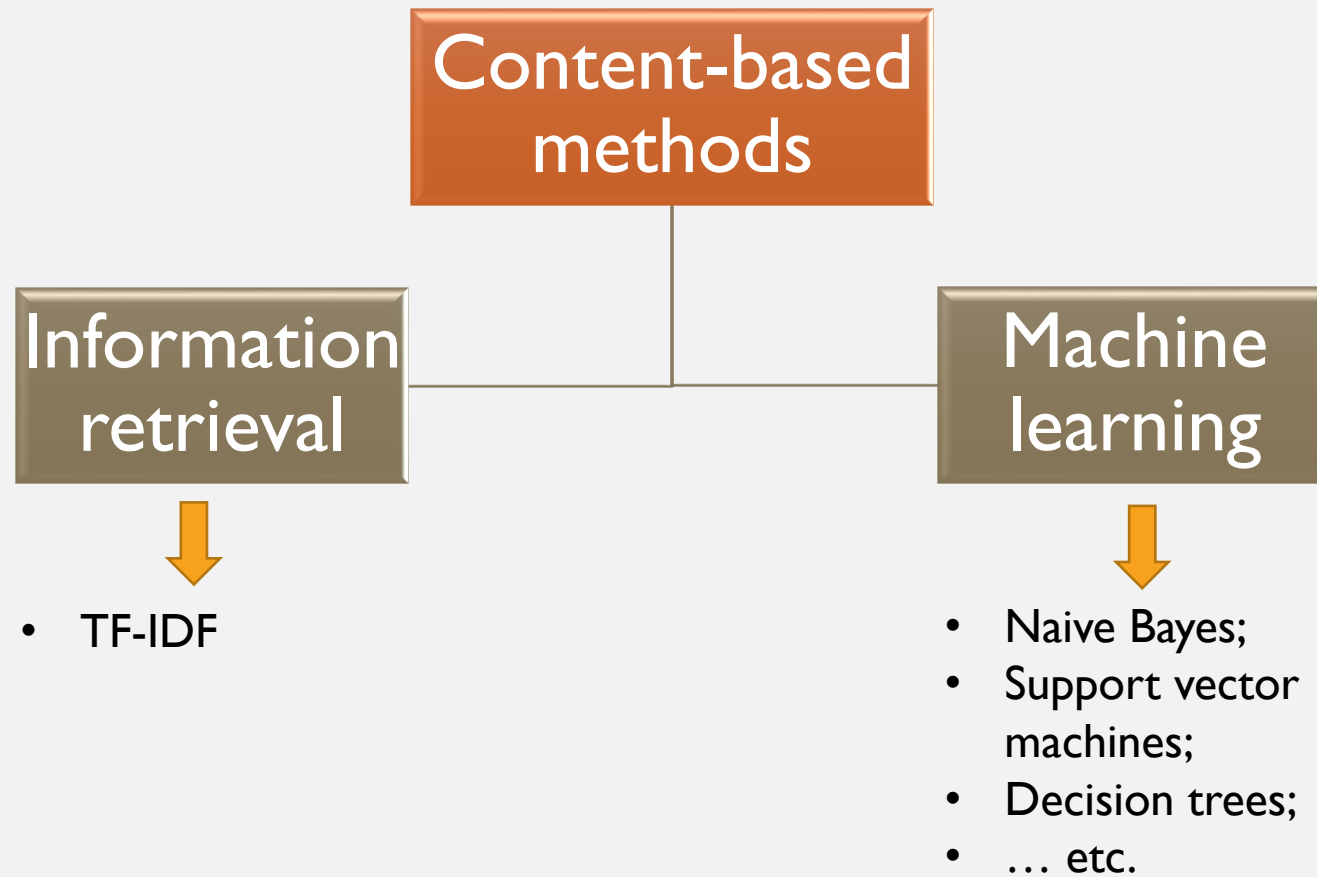
- When we search a group of keywords, the most basic idea will be to show all the items containing those keywords.
- Problem: Huge Pile of information !!
- **Example:**

if someone search for “The Recommender System”, there will be a large number of documents containing the keyword “**The**”. Same is for the keyword “**System**”.

So how do we select the most relevant item?



ALGORITHMS: CONTENT-BASED MODEL



ALGORITHMS: CONTENT-BASED MODEL



TF-IDF

Concept of TF-IDF:

- TF-IDF: **Term Frequency** times **Inverse Document Frequency**.
- **Term Frequency**
 - The number of times a given term appears in a specific document
 - Measure of the importance of the term in the particular document
 - **E.g.** how many times the keyword “The” appeared in the document OR how many times a tag is applied for a movie ?
- **Inverse Document Frequency**
 - How rare it is for a document to have this term or for a tag to be applied to the movie.
- High TF-IDF weight means high TF



ALGORITHMS: CONTENT-BASED MODEL



TF-IDF

Concept of TF-IDF:

- Given a keyword i and a document j
- $TF(i, j)$:
 - Term frequency of keyword i in document j ;
- $IDF(i)$:
 - Calculated as: $IDF(i) = \log \frac{N}{n(i)}$
 - N : Number of all recommendable documents;
 - $n(i)$: Number of documents from N in which keyword i appears.
- TF-IDF:
 - Is calculated as: $TF-IDF(i, j) = TF(i, j) * IDF(i)$.



ALGORITHMS: CONTENT-BASED MODEL

Pros	Cons
• No cold start problem • No need for usage data • No popularity bias, can recommend items with rare features • Can use content features to provide explanations	• Item content needs to be machine readable and meaningful • Easy to pigeonhole the user • Difficult to implement serendipity • Difficult to combine multiple item's features together



HYBRID APPROACHES

ALGORITHMS: HYBRID APPROACHES

Main idea

- Combines collaborative filtering and content-based filtering in order to use pros of one to address the cons of the other.
- **Example:**
 - CF algorithms suffer from new-item problems, i.e., they cannot recommend items that have no ratings/usage.
 - This does not limit content-based algorithms since the prediction for new items is based on their content (features) that are typically available when the new-item enters the system.
 - By creating a hybrid recommender, it is possible to overcome the cold-start and popularity bias problems.



ALGORITHMS: HYBRID APPROACHES

Different ways of combining two basic recommender algorithms

Method	Description
Weighted	Outputs from several techniques (in the form of scores or votes) are combined with different degrees of importance to offer final recommendations
Switching	Depending on situation, the system changes from one technique to another
Mixed	Recommendations from several techniques are presented at the same time
Feature combination	Features from different recommendation sources are combined as input to a single technique
Cascade	The output from one technique is used as input of another that refines the result
Feature augmentation	The output from one technique is used as input features to another
Meta-level	The model learned by one recommender is used as input to another



ALGORITHMS: HYBRID APPROACHES

Let's apply that on the previous example:

- Recommendation results obtained with used-based, item-based and content-based methods are already shown.
- Let's combine these three different algorithms to create a new hybrid recommender.
- Weighted method (from the table) will be used: it combines the output from several techniques, in this case three, with different degrees of importance (i.e. weight) to offer new set of recommendations.

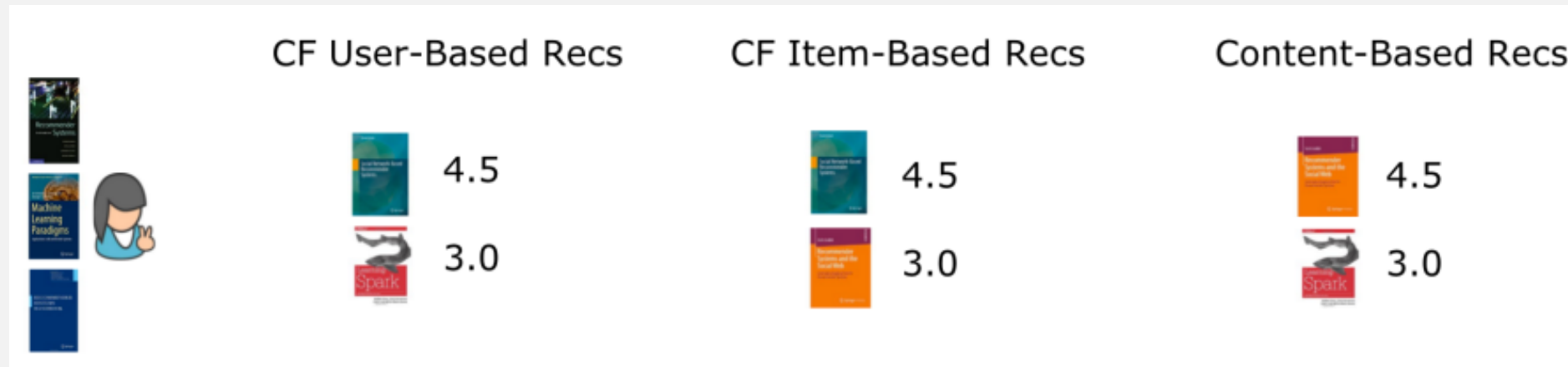


	4	3			5	
	5		4		4	
	4		5	3	4	
		3				5
		4				4
			2	4		5

ALGORITHMS: HYBRID APPROACHES

Let's apply that on the previous example:

- As a reminder, results previously obtained are different and are as following:



ALGORITHMS: HYBRID APPROACHES

Let's apply that on the previous example:

- Generate recommendations for the given user using a weighted hybrid recommender by putting:
 - 40% of the weight on user-based CF;
 - 30% on item-based CF ;
 - 30% on content-based filtering.
- Result shows that the user (1st one) would be recommended all three books that they have not rated yet, compared to getting just two book recommendations from the individual algorithms.

Hybrid (Weighted) Recs

$$(0.4 \times \text{UB CF}) + (0.3 \times \text{IB CF}) + (0.3 \times \text{CB}) =$$

					$(0.4 \times 4.5 + 0.3 \times 4.5) / (0.4 + 0.3) = 4.5$
					$(0.3 \times 3.0 + 0.3 \times 4.5) / (0.3 + 0.3) = 3.8$
					$(0.4 \times 3.0 + 0.3 \times 3.0) / (0.4 + 0.3) = 3.0$

ALGORITHMS: HYBRID APPROACHES

Pros	Cons
• Often outperforms CF and CB alone • No cold start problem • No popularity bias, can recommend items with rare features • Can implement serendipity, diversity	• Can be a lot of work to get the right balance



ALGORITHMS: HYBRID APPROACHES

More ideas to get an hybrid method:

- Another way of combining individual recommender algorithms is using **ensemble methods**.
- To do that, we **learn a function** (i.e. train an ensemble) for **how to mix** the results of **the different methods**.
- It is worth noting that usually ensembles combine not only different algorithms, but also different variations/models based on the same algorithm.

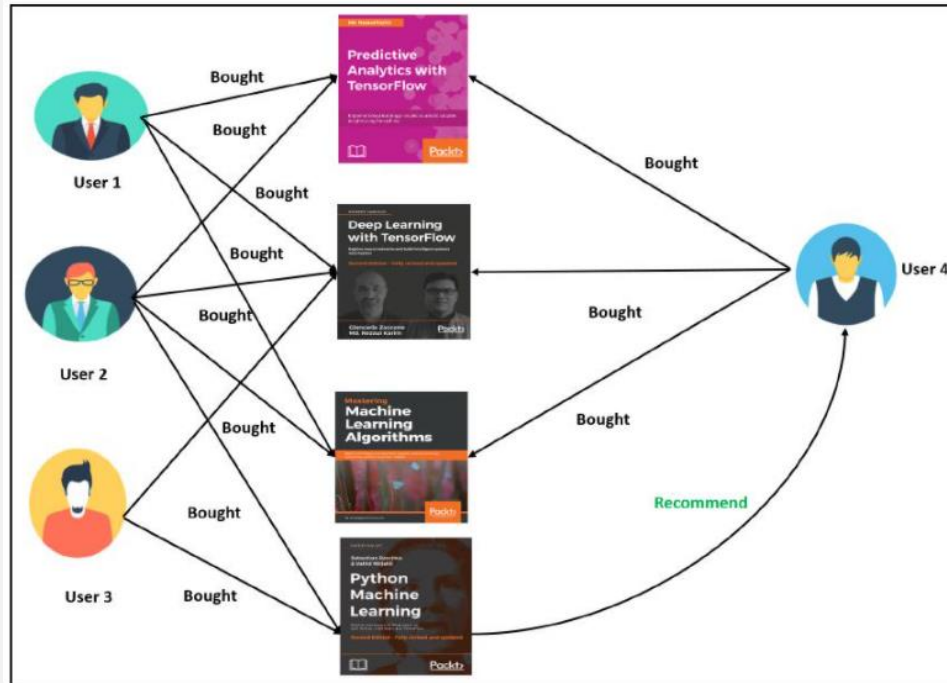
Example:

- Netflix ran a competition from 2006 to 2009 offering **\$1 mil** grand prize to the team that can generate recommendations that were **10% more accurate** than their recommender system at the time.
- **Winning solution:**
 - It consisted of over **100 different models** from more than **10 different algorithms** (popularity, neighbourhood methods, matrix factorisation, restricted boltzmann machines, regression and more), which were combined in an ensemble using gradient boosted decision trees.



OTHER FILTERING

ALGORITHMS: OTHER APPROACHES



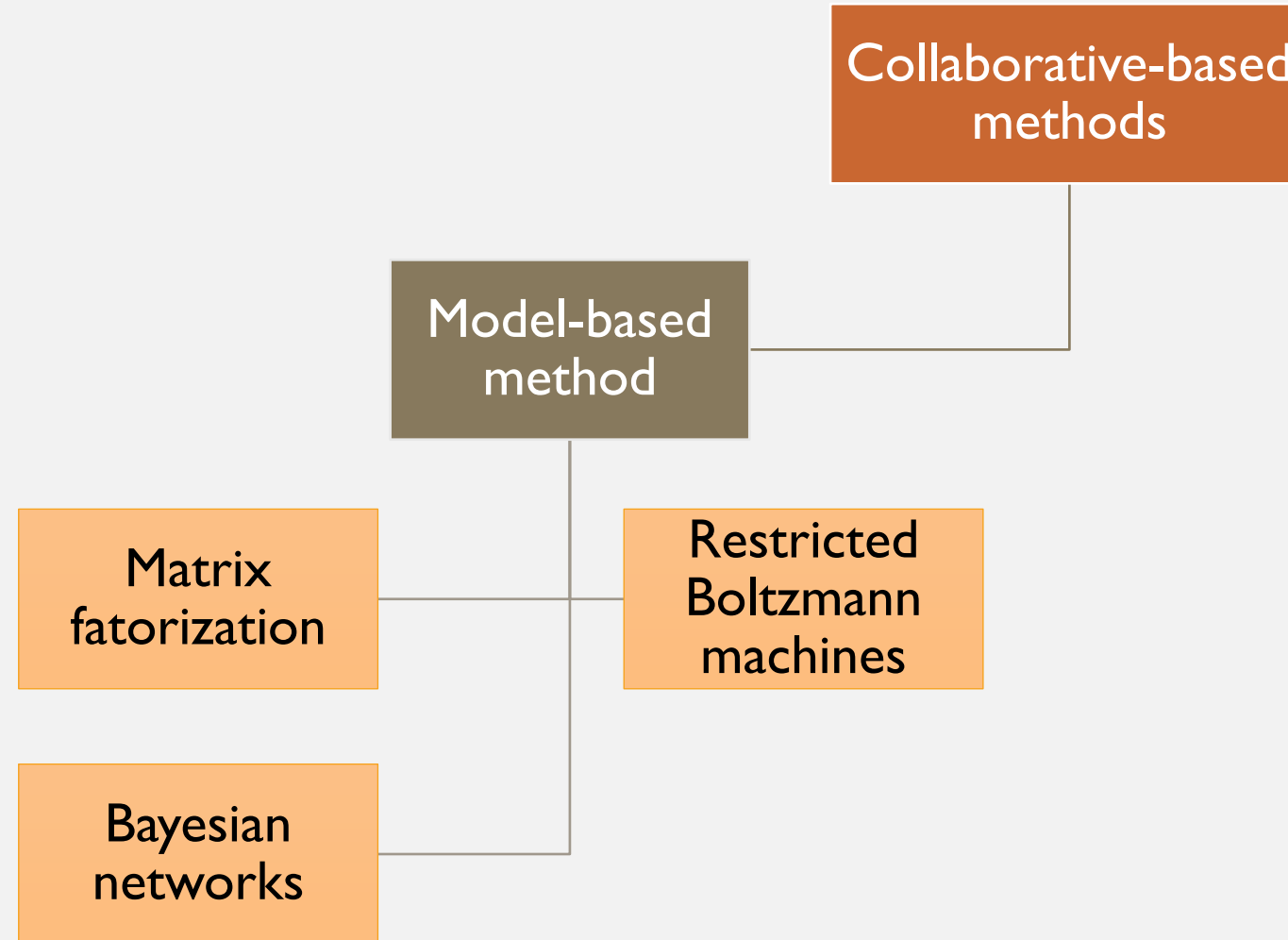
Another type of collaborative filtering.

A screenshot of a 'Quick Search' interface. It features a search bar with the placeholder text 'Search (optional)' and a dropdown menu set to 'All'. Below the search bar, there are three filter categories: 'Type of Play or Musical' with a dropdown set to 'Full Length Musical', 'Genre' with a dropdown set to 'Dramatic Comedy', and 'Total Cast' with three buttons: 'Large C', 'Medium', and 'Small C'. A green 'GO' button is located below the filters. At the bottom, there is a link that says 'Use Advanced Search'.

Knowledge-based filtering.

PREDICTION PROBLEM

ALGORITHMS: MODEL-BASED METHOD

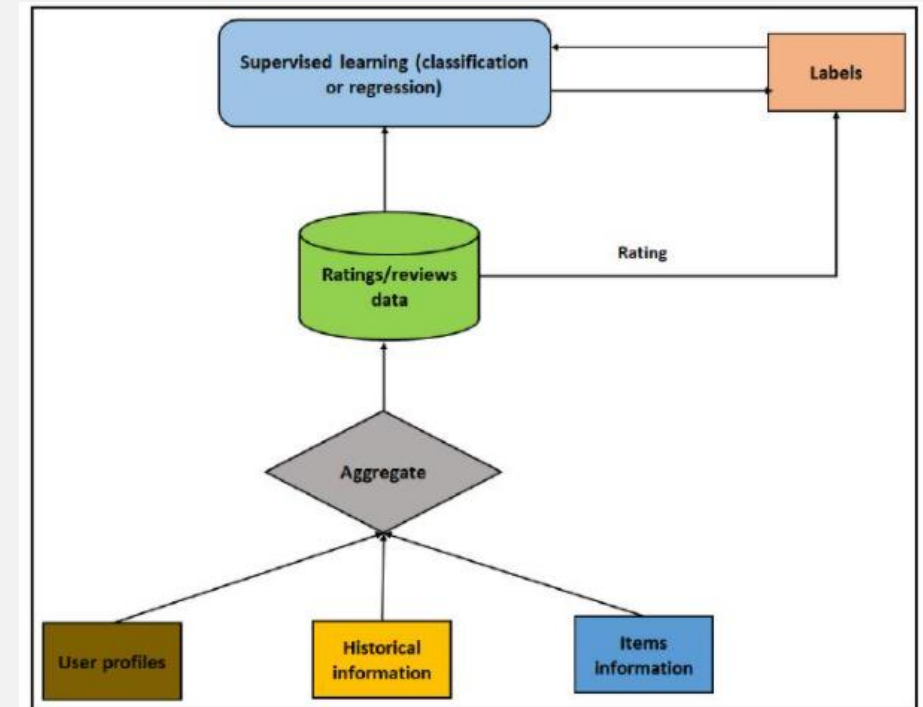


ALGORITHMS: CONTENT-BASED MODEL

→ Machine learning

Machine learning filtering:

- Based on classical machine learning techniques such as classification or regression.
- learns how to represent an item (book) I_j and a user U_i . Then, a separate feature matrix for both I_j and U_i are created before combining them as a feature vector. Then the feature vector is fed into a classification or regression model for the training.
- The ML model generates the label L_{ij} , which is the corresponding rating given by the user U_i on the item I_j



ALGORITHMS: CONTENT-BASED MODEL

→ Machine learning

Probabilistic methods:

- **Basic idea (simplistic version for illustration)**

- Given the user/item rating matrix
- Determine the probability that user Alice will like an item i
- Base the recommendation on such these probabilities.

- **Calculation of rating probabilities based on Bayes theorem**

- How probable is rating value « 1 » for item5 given Alice's previous ratings ?
- Corresponds to conditional probability $P(\text{item5}=1|X)$, where: $X = \text{Alice's previous ratings} = (\text{item1}=1, \text{item2}=3, \text{item3}=3, \dots)$
- Assumption: features are independent
- Can be estimated based on Bayes' Theorem

$$P(Y|X) = \frac{P(X|Y) \times P(Y)}{P(X)}$$

$$P(Y|X) = \frac{\prod_{i=1}^d P(X_i|Y) \times P(Y)}{P(X)}$$

Constant: Naive Bayes



ALGORITHMS: CONTENT-BASED MODEL

➡ Probabilistic method

Example:

	Item1	Item2	Item3	Item4	Item5
Alice	1	3	3	2	?
User1	2	4	2	2	4
User2	1	3	3	5	1
User3	4	5	2	3	3
User4	1	1	5	2	1

$X = (\text{Item1}=1, \text{Item2}=3, \text{Item3}= \dots)$

$$\begin{aligned} P(X|\text{Item5} = 1) &= P(\text{Item1} = 1|\text{Item5} = 1) \times P(\text{Item2} = 3|\text{Item5} = 1) \\ &\times P(\text{Item3} = 3|\text{Item5} = 1) \times P(\text{Item4} = 2|\text{Item5} = 1) \\ &= \frac{2}{2} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} \approx 0.125 \end{aligned}$$

$$\begin{aligned} P(X|\text{Item5} = 2) &= P(\text{Item1} = 1|\text{Item5} = 2) \times P(\text{Item2} = 3|\text{Item5} = 2) \\ &\times P(\text{Item3} = 3|\text{Item5} = 2) \times P(\text{Item4} = 2|\text{Item5} = 2) \\ &= \frac{0}{0} \times \dots \times \dots \times \dots = 0 \end{aligned}$$



ALGORITHMS: MODEL-BASED METHOD

- Model-based CF approaches can help overcome some of the limitations of-based methods.
- Model-based approaches use the ratings to learn a predictive model based on which predictions on new items are made.

General idea

- The general idea is to model the user-item interactions using **machine learning** algorithms which find patterns in the data.
- There are a number of different algorithms that can be used for building the models based on which to make predictions:
 - Clustering;
 - Bayesian networks
 - Matrix factorization
 - Restricted Boltzmann machines;
 - ...etc.



ALGORITHMS: MODEL-BASED METHOD

The utility matrix

- Often called a user-item where each row represents a user and each column represents an item (book), while a cell represents the corresponding rating given by the user to that item.
- Also, it contains many empty cells that indicate users have not given any ratings for those items.
- In practice, the utility matrix is very sparse because a large number of cells are empty.

userid	4	5	10	14	15	18	19	26	31	34	...	283199	283204	283206	283208	283210	283215	283219	283222	283224
movieid																				
1	4.0	NaN	5.0	4.5	4.0	NaN	NaN	NaN	5.0	NaN	...	5.0	NaN	NaN	4.5	NaN	4.0	4.0	NaN	NaN
2	4.0	NaN	NaN	4.0	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN	NaN	4.0	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	5.0	NaN	NaN	NaN	4.0
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	2.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	5.0	NaN	NaN	NaN	NaN
6	4.5	NaN	NaN	NaN	NaN	3.0	4.0	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	5.0	NaN	NaN	NaN	NaN
8	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9	NaN	NaN	NaN	NaN	NaN	NaN	4.0	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
10	4.0	NaN	NaN	NaN	NaN	3.0	4.0	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	3.0
11	3.5	NaN	NaN	NaN	NaN	NaN	4.0	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	3.0	NaN	NaN	NaN
12	NaN	NaN	NaN	NaN	NaN	NaN	3.0	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
13	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
14	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	3.0
15	NaN	NaN	NaN	NaN	NaN	NaN	3.0	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

	Python Machine Learning	Statistics for Machine Learning	Deep Learning with TensorFlow	Mastering Machine Learning Algorithms	Scala and Spark for Big Data Analytics	Predictive Analytics with TensorFlow
	7	6		?	5	
	8		7		6	
	6		6	5	7	
		7				8
		8				7
			5	4		5



ALGORITHMS: MODEL-BASED METHOD

The utility matrix: Example

Suppose there are 1 million users (n) and only 10,000 items (movies, m), which is $10,000,000 * 10,000$ or 10^{11} , a very large number. Now, even if a user has rated 10 books, this means that the total number of given ratings will be $10 * 1 \text{ million} = 10^7$. The sparsity of this matrix can be calculated as follows:

$$S_m = \text{Number of empty cells} / \text{Total number of cells} = (10^{10} - 10^7) / 10^{10} = 0.9999$$

This means 99.99% of the cells will still be empty.



ALGORITHMS: MODEL-BASED METHOD

Variants

Purchasing matrix

Item

User

0	0	1	0	1	0	0	0	0	0	0
1	0	0	0	0	1	1	0	0	1	0
1	1	0	0	0	0	0	0	1	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1	0	0	0
0	0	1		0	0	0	0	1	0	0
0	1	1	1	0	0	0	0	1	1	1

0 can mean that:

- The user does not appreciate the item
- The user does not know the item (Or does not yet gave his opinion about a specific item)

Rating matrix

Movies

Spectators

?	?	4	?	1	?	?	?	?	?	?
3	?	?	?	?	4	3	?	?	2	?
1	5	?	?	?	?	?	?	5	?	?
?	2	?	?	?	?	?	?	?	?	1
?	?	?	2	?	?	?	2	?	?	?
?	?	1	?	?	?	?	?	1	?	?
?	5	3	4	?	?	?	?	1	1	4

? means that the spectator opinion is unknown



ALGORITHMS: MODEL-BASED METHOD

Matrix factorisation

- What matrix factorization does is to come up with two smaller matrices, one representing users and one representing items, which when multiplied together will produce roughly this matrix of ratings, ignoring the 0 entries.
- It got pretty popular after the Netflix prize competition.

Latent factors / features

- Features that are not directly observed but are rather inferred from other variables that are observed (directly measured).
- For example, the latent factor could be “Movie Genre”, such as action, romantic, comics, adventure, etc.

	Feature 1	Feature 2
User 1	?	?
User 2	?	?
User 3	?	?
User 4	?	?
User 5	?	?

 $\times$

	Item 1	Item 2	Item 3	Item 4	Item 5
Feature 1	?	?	?	?	?
Feature 2	?	?	?	?	?

 $=$

	Item 1	Item 2	Item 3	Item 4	Item 5
User 1	0?	3	0?	3	0?
User 2	4	0?	0?	2	0?
User 3	0?	0?	3	0?	0?
User 4	3	0?	4	0?	3
User 5	4	3	0?	4	0?



ALGORITHMS: MODEL-BASED METHOD

Matrix factorisation

- Model learns to factorize rating matrix into user and movie representations, which allows model to predict better personalized movie ratings for users (**PCA** is used).
- With matrix factorization, less-known movies can have rich latent representations as much as popular movies have, which improves recommender's ability to recommend less-known movies

The diagram illustrates the process of matrix factorization. It shows a 'Rating Matrix' on the left, which is a 4x4 grid with rows labeled A, B, C, D and columns labeled W, X, Y, Z. The values are: A (W: 4.0, X: 4.5, Y: 2.0, Z: 1.0), B (W: 4.0, X: 5.0, Y: 3.5, Z: 2.0), C (W: 4.0, X: 3.5, Y: 4.0, Z: 1.0), and D (W: 4.0, X: 3.5, Y: 4.0, Z: 1.0). This matrix is equal to the product of a 'User Matrix' (4x2) and an 'Item Matrix' (2x4). The 'User Matrix' has rows A, B, C, D with values: A (1.2, 0.8), B (1.4, 0.9), C (1.5, 1.0), and D (1.2, 0.8). The 'Item Matrix' has columns W, X, Y, Z with values: W (1.5, 1.7), X (1.2, 0.6), Y (1.0, 1.1), and Z (0.8, 0.4). The matrices are separated by an equals sign and a large 'X' symbol.

		Item			
		W	X	Y	Z
User	A		4.5	2.0	
	B	4.0		3.5	
	C		5.0		2.0
	D		3.5	4.0	1.0

Rating Matrix

=

	User Matrix	
	A	1.2 0.8
	B	1.4 0.9
	C	1.5 1.0
D	1.2 0.8	

X

		Item Matrix			
		W	X	Y	Z
	A	1.5	1.2	1.0	0.8
	B	1.7	0.6	1.1	0.4

ALGORITHMS: MODEL-BASED METHOD

Matrix factorisation

- In the sparse user-item interaction matrix, the predicted rating user u will give item i is computed as:

$$\tilde{r}_{ui} = \sum_{f=0}^{n \text{ factors}} H_{u,f} W_{f,i}$$

where H is user matrix, W is item matrix

- Rating of item i given by user u can be expressed as a dot product of the user latent vector and the item latent vector.
- Optimize those values such that the error is minimized.



ALGORITHMS: MODEL-BASED METHOD

Matrix factorisation: error minimization

- The two matrices are selected such that the error for the users-book pairs (in the case of known rating) gets minimized.

$$\arg \min_{H, W} \|R - \tilde{R}\|_F + \alpha \|H\| + \beta \|W\|$$

where H is user matrix, W is item matrix

- Once we have an objective function, we just need a training routine (eg, gradient descent) to complete the implementation of a matrix factorization algorithm : **Funk SVD** (Simon Funk).
- **Algorithm example:**
 - **ALS (Alternating Least Square)** : first fills the user matrix with random values (between 1 and 10, for example) and then optimizes those values such that the error is minimized.



ALGORITHMS: MODEL-BASED METHOD

Factorization techniques

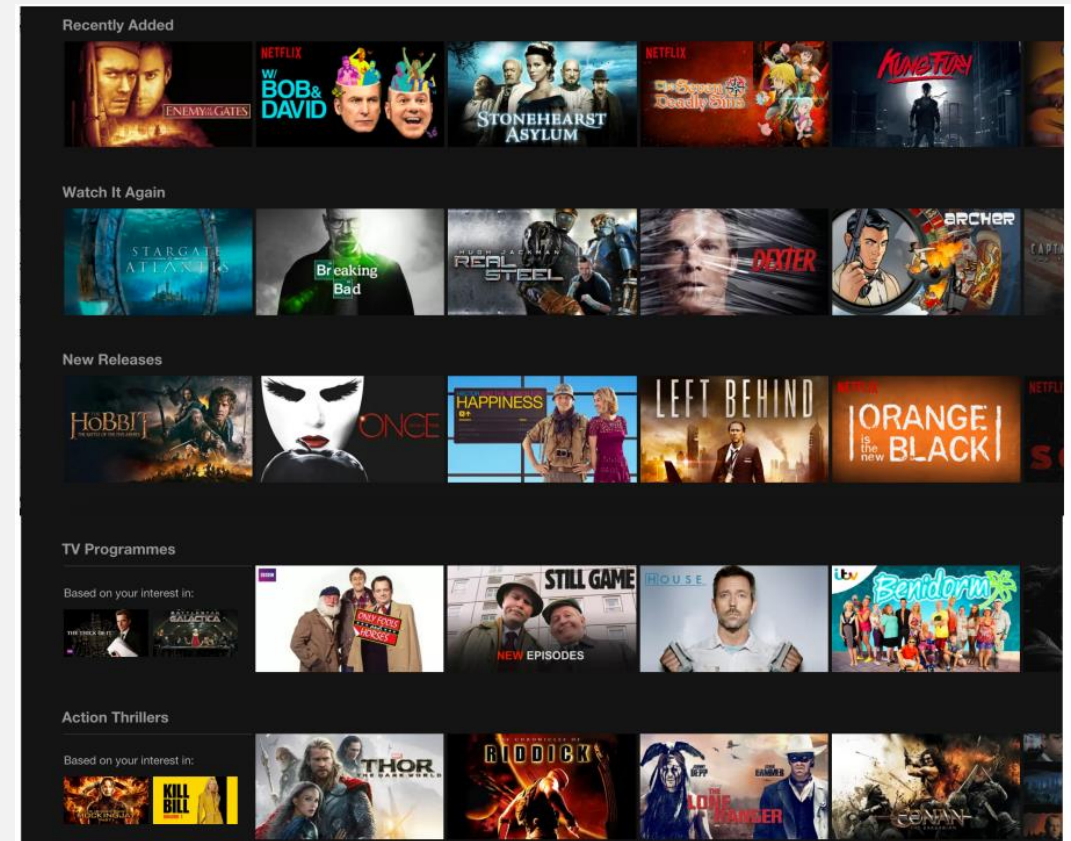
- Computational cost
- Explainability
- Combine with neighborhood systems (Koren 2008)
- Restricted Boltzmann machines
- Bayesian networks



NETFLIX RECOMMENDER SYSTEM

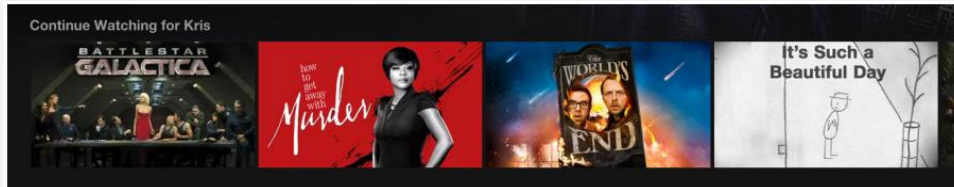
RECOMMENDER SYSTEM IN NETFLIX

- Netflix manages a large collections of movies and television programs. It demonstrates how to successfully commercialize recommender systems.

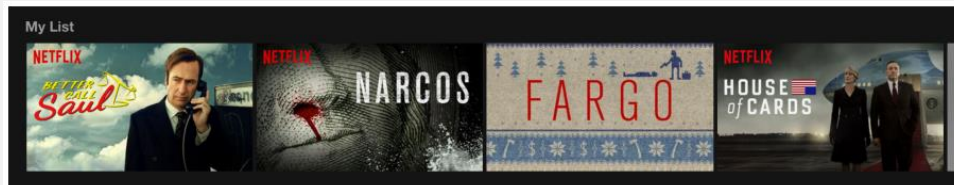


RECOMMENDER SYSTEM IN NETFLIX

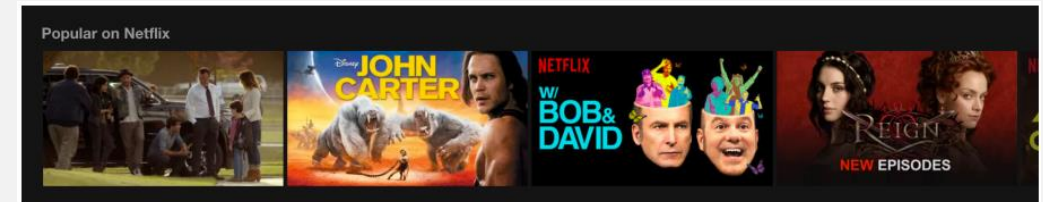
- The **first** carousel is filled with movies that the user has recently watched. It's a history of what he last watched: **it plays an important part in building a relationship with the users.**



- The **second** carousel's content is not provided by a recommender algorithm but it's a personal list of movies that the user has manually curated.



- The **third** carousel recommends movies that are popular in the Netflix viewing community. Popularity is a good, solid algorithm for generating recommendations, but Netflix personalizes the content here so not all users will see the same list of popular movies.

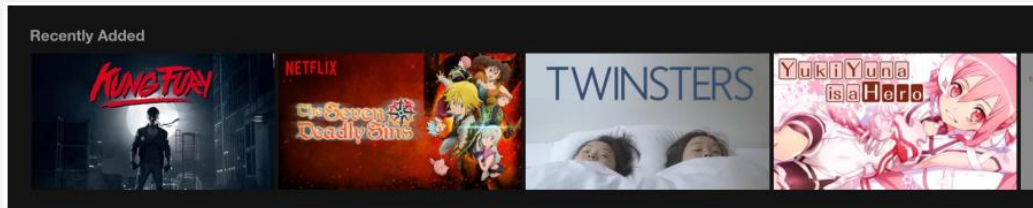


- The next carousel recommends movies that are currently trending. Again, they are personalized so won't be the same for every user.

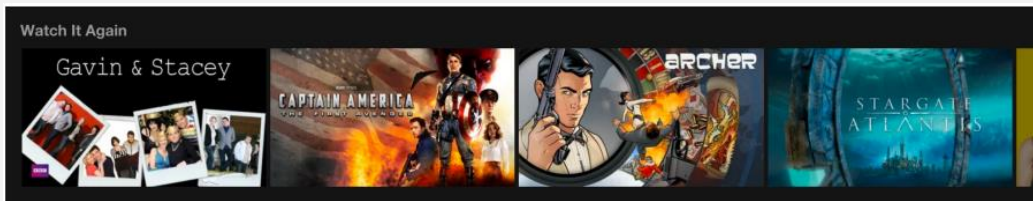


RECOMMENDER SYSTEM IN NETFLIX

- The next carousel recommends movies that have been recently added that it thinks will appeal to the user.



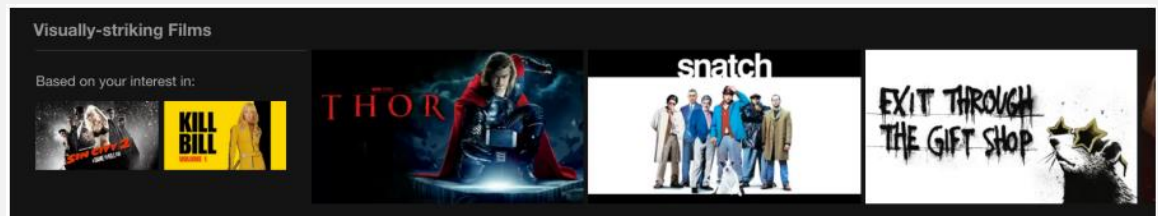
- Similar to the 'Continue Watching' carousel, that was first described, Netflix also provides a carousel with movies that the user has already watched but in different order. that attempts to bias it towards most recent user interests.



- Netflix also provides recommendations based on newly released content. This is different from 'Recently Added' movies as it's probably both recently added and recently released. Again, these are personalized to the user interests.



- It's really interesting to see the title given to the carousel in 'Visually-striking Films'. This description is likely to be hand-written but since the lists of movies are personalized, there will certainly be some automation in linking the descriptions with movies.



EVALUATION

EVALUATION

- What is a good recommendation?
- What is a good recommendation strategy?
- What is a good recommendation for my business?

EVALUATION

We can distinguish 6 main families:

- rating and usage prediction accuracy
- user- and item-space coverage
- user and item temperature
- ranking
- ‘advanced’ metrics
- online metrics

SOME DEFINITIONS

- **Coverage:** For how many users can we make recommendations? How many catalog items are ever recommended?
- **Diversity and Novelty:** Avoiding monotone lists
- **Serendipity:** Unexpected and surprising items might be valuable
- **Familiarity:** Give the user the impression of understanding his/her needs
- **Biases:** Does the recommender only recommend popular items and blockbusters?



ATTACKS ON COLLABORATIVE FILTERING

ATTACKS FOR COLLABORATIVE FILTERING

Some naive strategies

- Create fake users
- Rate product highly
- Only works for aggregate
- More elaborate attacks for collaborative filtering
- Try to inject profiles close to many users



ATTACKS FOR COLLABORATIVE FILTERING

Some naive strategies



ATTACKS FOR COLLABORATIVE FILTERING

Countermeasures

- Hybrid systems
- Cost of injection (captchas)
- Statistical attack detection:
 - Groups of users
 - Evolution of item ratings
 - Machine learning to detect fake profiles/reviews

